

CaseFile AI

Technical Report - Interactive Agentic RAG Investigation

1. Overview

CaseFile AI is an investigation-style RAG system built around Arthur Conan Doyle's public-domain story *The Adventure of the Norwood Builder*. I wanted the app to behave less like a one-shot chatbot and more like an actual case investigation: the user can search evidence, interrogate people, inspect a graph and timeline, make an initial prediction, and then compare it with an Investigator agent and a separate adversarial Fact-Checker.

A major design choice was to separate *what a source says* from *what the evidence establishes*. Extracted evidence is therefore labelled verified, unverified, disputed, or misleading. This prevents witness statements, police theories, or suspect claims from being treated as objective facts just because they appear in the corpus.

2. Data and evidence pipeline

The source story is split into 16 source-preserving document chunks with stable DOC IDs. Gemini is used for structured extraction, but extracted excerpts are retained only if they match the original text. A conservative canonicalization stage merges aliases without fuzzy identity matching.

- 30 canonical entities, 87 evidence items, 26 relationships, and 10 timeline events.
- 20 evidence items are verified and 67 are unverified.
- Evidence keeps the source document, source excerpt, speaker, type, status, linked entities, and support/contradiction labels.
- The final NetworkX MultiDiGraph contains 152 nodes, 470 edges, and no isolated canonical entities.

3. Retrieval architecture

Retrieval is deliberately multi-stage because different query types behave differently. BM25 is strong for exact names and phrases, MiniLM helps with paraphrases, and graph-aware retrieval contributes structural evidence for relational queries. These candidate lists are generated in parallel, fused with Reciprocal Rank Fusion (RRF), and finally reranked by a CrossEncoder.

BM25 + MiniLM + Graph-aware retrieval -> RRF fusion -> CrossEncoder reranking

Models: sentence-transformers/all-MiniLM-L6-v2 for semantic retrieval and cross-encoder/ms-marco-MiniLM-L-6-v2 for final reranking.

4. Agent design and self-correction

The Investigator uses a bounded loop rather than producing a final answer immediately. It forms a working theory, retrieves evidence, judges sufficiency, identifies gaps, reformulates the search, and retries. If normal retrieval is still insufficient, it runs a wider coverage sweep and a final verified-evidence safety pass before producing a cited conclusion.

The Fact-Checker performs its own adversarial retrieval instead of simply reading the Investigator answer. It searches for contradictions, alternative explanations, timeline conflicts, weaknesses in the theory, and evidence supporting a different interpretation.

Important debugging lesson: during testing, the Investigator produced a fully grounded answer but still missed a decisive verified observation. This exposed the difference between citation faithfulness and completeness. I therefore separated the final audit into (1) citation grounding and (2) evidence coverage.

5. Timeline and graph reasoning

The evidence graph contains entity, evidence, document, hypothesis, event, and time nodes. The timeline also separates the order in which information enters the investigation from the relative time described in the story. Each event has core evidence and optional context evidence, and its status is derived from the core evidence only. This avoids downgrading a directly observed event because surrounding testimony is uncertain.

6. Evaluation

I used a small manually curated six-query benchmark as a regression/sanity check. It is not meant to be a statistically strong evaluation dataset, but it is useful for catching retrieval regressions during development.

Retriever	Hit@1	Hit@3	Hit@5	MRR@5
BM25	50.0%	100.0%	100.0%	69.4%
MiniLM semantic	83.3%	83.3%	83.3%	83.3%
Hybrid RRF	66.7%	100.0%	100.0%	83.3%
Graph-aware RRF	66.7%	100.0%	100.0%	80.6%
Production + CrossEncoder	100.0%	100.0%	100.0%	100.0%

Result: the full production pipeline reached 100% Hit@1/Hit@3/Hit@5/MRR@5 on this six-query set. I do not claim that graph retrieval alone improved accuracy - in fact, its standalone MRR was slightly lower than plain hybrid RRF. The best performance came from the full candidate-generation + CrossEncoder pipeline.

Parallel retrieval test: three artificial 0.30 s retrieval arms completed in about 0.305 s instead of the ~0.90 s expected sequentially, confirming that the independent retrieval arms actually execute concurrently.

7. Extra features implemented

- CrossEncoder reranking and graph-aware retrieval
- contradiction detection and adversarial Fact-Checker retrieval
- temporal reasoning with a structured investigation timeline
- parallel retrieval
- agent execution traces
- automatic grounding and coverage audits
- visual evidence graph

8. Limitations and conclusion

The project currently supports one case and does not keep persistent cross-session agent memory. The benchmark is intentionally small, and some timeline information is relative because the source does not provide exact timestamps. The reasoning layer also still depends on an external LLM, so the validation and coverage guards reduce failure risk rather than eliminating it.

Overall, I focused on making one case traceable and defensible instead of adding features only for feature count. The final system combines hybrid RAG, structured evidence provenance, graph and temporal reasoning, bounded self-correction, adversarial checking, and explicit uncertainty handling in a single investigation workflow.